



UNIVERSIDAD INTERNACIONAL DEL ECUADOR

Carrera de Ingeniería en Ciberseguridad

Modalidad en Línea

Proyecto Integrador

Informe del proyecto Juego del Ahorcado

Asignatura: Lógica de programación
Estudiante: Carlos Lozada
Período Académico: 2026-I
Fecha: Junio, 2026

Quito — Ecuador

2026

1. Introducción

En el ámbito de la ingeniería de software moderna y la ciberseguridad, el desarrollo de aplicaciones eficientes y seguras exige la adopción de metodologías de diseño estructuralmente robustas. El paradigma educativo actual fomenta el pensamiento computacional a través de la creación de herramientas interactivas y lúdicas que, a pesar de su aparente simplicidad conceptual, demandan una rigurosa aplicación de principios lógicos fundamentales.

El presente proyecto académico aborda el diseño, la estructuración y la implementación en código del clásico "Juego del Ahorcado" en un entorno de consola, utilizando el lenguaje de programación Python. Más allá de la mecánica recreativa intrínseca del juego, esta investigación se enfoca en resolver las limitaciones de los desarrollos monolíticos tradicionales mediante la implementación de una arquitectura modular basada en la división de responsabilidades. Para ello, el sistema ha sido fragmentado en un patrón arquitectónico de tres capas independientes (Presentación, Control y Modelo), garantizando con ello la mantenibilidad del código, facilitando las pruebas unitarias y emulando las metodologías de desarrollo seguro que se aplican en la construcción de infraestructuras de software críticas.

2. Objetivos

2.1 Objetivo General

Diseñar e implementar un sistema interactivo del "Juego del Ahorcado" en consola empleando el lenguaje Python, estructurado bajo una arquitectura de tres capas y aplicando de manera integral los conocimientos lógicos, modulares y de estructuras de datos adquiridos a lo largo de la asignatura de Lógica de Programación.

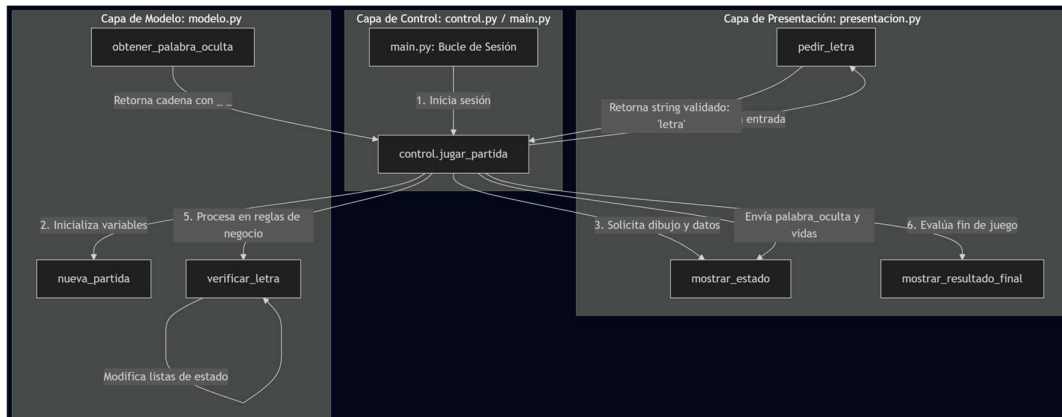
2.2 Objetivos Específicos

- Descomponer el sistema de software en tres módulos claramente diferenciados (`presentacion.py`, `control.py` y `modelo.py`) para aislar la lógica de negocio de las interfaces visuales de usuario.
- Integrar de forma correcta las estructuras de control lógicas condicionales y los bucles iterativos repetitivos para gobernar eficazmente el ciclo de vida de una partida y de la sesión global del usuario.
- Implementar mecanismos robustos de entrada/salida de datos y manipulación de archivos para la carga dinámica de palabras desde un almacén persistente externo.
- Analizar y reflexionar críticamente sobre la correlación existente entre las buenas prácticas de la lógica de programación estructurada, la modularidad y los pilares fundamentales de la ciberseguridad.

3. Marco Teórico y Diseño Estructural

3.1 Arquitectura de Software en Tres Capas

El diseño de sistemas complejos requiere el aislamiento de funciones para mitigar el acoplamiento y potenciar la cohesión. La arquitectura en tres capas es un patrón de diseño que distribuye la carga operativa del sistema en tres niveles lógicos independientes:

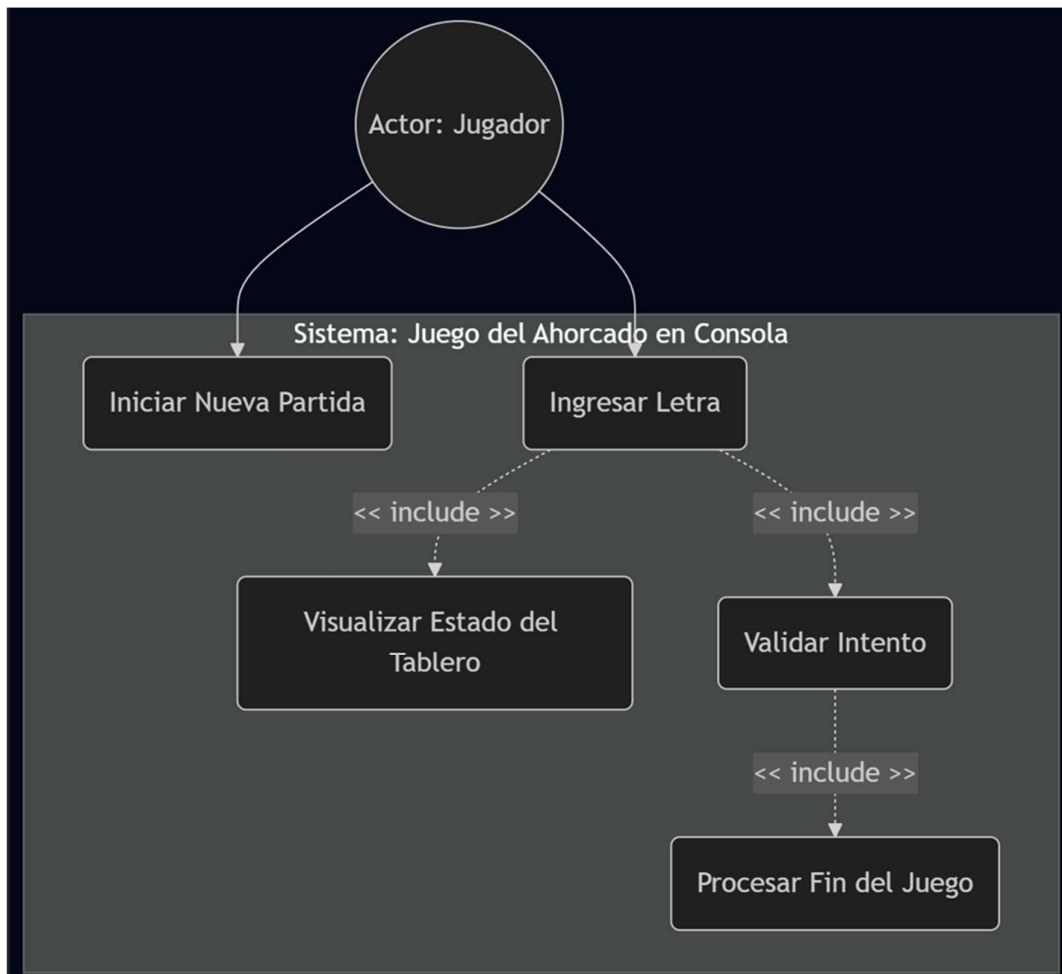


- **Capa de Presentación:** Es la frontera del sistema con el exterior. Su única responsabilidad es la captura de datos primitivos provistos por el usuario, su validación sintáctica básica y la renderización de la información en un formato comprensible. No conoce las reglas del juego ni manipula estados lógicos internos.
- **Capa de Control:** Actúa como un orquestador o intermediario de comunicaciones. Recibe las peticiones de la capa superior, invoca los métodos pertinentes del modelo para procesar las reglas de negocio y, tras recibir el estado resultante, instruye a la capa visual sobre qué elementos debe refrescar. Contiene el ciclo algorítmico principal (*game loop*).
- **Capa de Modelo:** Representa el núcleo computacional y semántico de la aplicación. Es el componente encargado de almacenar las variables de estado en memoria, implementar las restricciones algorítmicas (reglas de negocio), realizar cálculos lógicos y gestionar el acceso a los datos persistentes.

3.2 Diagramas de Funcionalidad

El flujo del sistema y las interacciones del usuario se modelaron previamente para garantizar que la traducción al código fuente reflejara con exactitud los requerimientos lógicos establecidos.

3.2.1 Diagrama de Casos de Uso



El sistema identifica al **Jugador** como el actor principal de la sesión en consola. El entorno limita el alcance de las interacciones directas del actor a dos eventos raíz: **Iniciar Nueva Partida** e **Ingresar Letra**. Internamente, el caso de uso "Ingresar Letra" genera relaciones de inclusión obligatoria (<<include>>) hacia las funciones del sistema encargadas de **Visualizar Estado del Tablero**, **Validar Intento** y, consecuentemente, **Procesar Fin del Juego**, encapsulando la operatividad sin exponer variables internas.

3.2.2 Diagrama de Flujo (Ciclo del Juego)

El comportamiento dinámico se rige por un autómata que se inicializa invocando el método `modelo.nueva_partida`. Tras configurar el estado inicial, se ingresa a un condicional iterativo evaluado por las funciones del modelo `jugador_gano` y `jugador_perdio`. En caso de persistir el juego, se ejecutan secuencialmente operaciones aritméticas de cálculo de errores, llamadas procedimentales de renderizado, captura y validación de caracteres, control de redundancia (letras repetidas) y, finalmente, la mutación de las listas de estado en función de la coincidencia del carácter con la cadena secreta.

4. Desarrollo del Software y Relación con los Contenidos de la Asignatura

El código fuente adjunto en el repositorio evidencia una traducción sistemática de la teoría algorítmica a construcciones lógicas explícitas de Python, organizadas de la siguiente manera:

4.1 Unidad 1 y 2: Estructuras Lógicas Condicionales y Repetitivas

El control del flujo de ejecución del programa depende estrictamente de la evaluación de expresiones booleanas y ciclos indeterminados:

- **Ciclos de Vida Dinámicos:** En `main.py`, se implementa un bucle infinito `while True` que encapsula la sesión completa del software, delegando al método condicional `control.preguntar_revancha()` la ruptura explícita del ciclo mediante la sentencia `break`. Asimismo, el ciclo del juego en `control.py` corre bajo la condición unificada `while not modelo.jugador_gano() and not modelo.jugador_perdio()`; asegurando que la iteración se detenga inmediatamente al cumplirse las condiciones de parada.
- **Evaluación Bifurcada:** La verificación de caracteres en `modelo.py` utiliza la estructura condicional `if letra in palabra_secreta` para discriminar el curso de acción de las variables de estado. Si la evaluación resulta verdadera, se altera la lista de aciertos; de lo contrario, se ejecuta una operación de decremento unitario sobre la variable cuantitativa `vidas` (`vidas = vidas - 1`).

4.2 Unidad 3: Colecciones de Datos y Persistencia en Archivos

El programa mitiga el acoplamiento duro de datos mediante el almacenamiento del vocabulario en un archivo de texto independiente (`data/prueba.txt`) y su manipulación a través de estructuras secuenciales dinámicas:

- **Manejo de Streams de Archivos:** La función `modelo.cargar_palabras(ruta)` realiza la apertura del recurso mediante la función primitiva `open` bajo codificación `utf-8`, garantizando la lectura segura de caracteres especiales. Utiliza un bucle `for linea in archivo` para recorrer secuencialmente las líneas, aplicando técnicas de sanitización como `.strip().lower()` para eliminar espacios en blanco residuales y saltos de línea antes de inyectar las cadenas en la memoria de trabajo.
- **Estructuras de Almacenamiento Indexado (Listas):** El estado interno se monitorea a través de listas dinámicas (`letras_correctas` y `letras_incorrectas`). La verificación de redundancia implementada en la función `letra_ya_fue_usada(letra)` se ejecuta evaluando la pertenencia lógica del elemento con el operador `in` sobre ambas colecciones unificadas por el operador lógico `or`.

4.3 Unidad 4: Modularidad, Ámbito de Variables y Funciones

El diseño de software robusto exige limitar el alcance (*scope*) de los identificadores para evitar efectos secundarios imprevistos:

- **Encapsulamiento y Variables Globales:** Dentro de `modelo.py`, las funciones que mutan el estado del juego hacen uso explícito de la palabra clave `global` (`global palabra_secreta, letras_correctas, letras_incorrectas, vidas`). Esto delimita el ámbito de modificación y asegura que las funciones de lectura como `jugador_gano()` operen en un entorno de solo lectura, protegiendo la integridad de los datos.
- **Abstracción Procedimental:** La capa de presentación (`presentacion.py`) expone interfaces abstractas como `mostrar_estado` y `pedir_letra`. La función `pedir_letra` implementa un ciclo de validación estricto que restringe la salida de datos únicamente si la cadena ingresada satisface simultáneamente las condiciones `len(letra) == 1` y `letra.isalpha()`, aislando a las capas superiores de entradas maliciosas o erróneas.

5. Integración y Explicación del Código Desarrollado

La arquitectura modular se consolida mediante el archivo raíz `main.py`, el cual actúa como el punto de entrada lógico (*Entry Point*) del sistema. A nivel de integración de componentes, la interacción ocurre mediante un esquema de llamadas procedimentales estructuradas:

1. **Inicialización:** `main.py` invoca a `modelo.cargar_palabras()`, reteniendo la lista resultante en una variable de ámbito local. Posteriormente, transfiere el control de la ejecución al módulo intermedio llamando a `control.jugar_partida(lista_palabras)`.
2. **Sincronización:** El controlador toma el mando, invoca inmediatamente a `modelo.nueva_partida()` para resetear los acumuladores e ingresa al ciclo principal. En cada iteración, el controlador calcula la diferencia aritmética de vidas para determinar el índice de error y solicita a la presentación renderizar la pantalla enviando parámetros por valor (`pantalla.mostrar_estado(...)`).
3. **Validación y Retorno:** Tras desplegar la interfaz gráfica ASCII, la presentación retiene el flujo de ejecución en su bucle interno hasta que el jugador proporcione un carácter válido, retornándolo como un objeto tipo *string*. El controlador intercepta este retorno, valida la no-repetición y delega la lógica de negocio al modelo mediante `modelo.verificar_letra(letra)`. Este ciclo cerrado se repite de manera síncrona hasta la conclusión de la partida.

6. Reflexión sobre el Impacto de la Tecnología en la Sociedad y la Ciberseguridad

En el contexto de la evolución tecnológica contemporánea, la automatización y la optimización de procesos digitales dependen directamente de la calidad estructural del software subyacente. El análisis detallado de la arquitectura de este proyecto revela una conexión unívoca con los principios fundamentales de la **Ingeniería en Ciberseguridad**.

La división estricta del sistema en capas independientes constituye la base del principio de **Defensa en Profundidad** y del **Aislamiento de Privilegios**. Al impedir que la interfaz gráfica (Presentación) acceda directamente a las variables que determinan el estado del juego (Modelo), se emula el comportamiento de un sistema seguro donde las entradas del usuario nunca deben poseer control directo sobre la memoria o las reglas de negocio de una aplicación corporativa.

Asimismo, los algoritmos de sanitización implementados en la función `pedir_letra()` (restringiendo la longitud de la cadena y forzando la validación alfabética) representan la primera línea de defensa contra vulnerabilidades críticas de la industria, tales como desbordamientos de búfer (*Buffer Overflow*) e inyecciones de código. En el ecosistema socio-tecnológico actual, donde la infraestructura crítica, la banca y la gestión de identidades dependen de sistemas automatizados, la internalización de estas prácticas de lógica computacional y desarrollo seguro desde las etapas formativas de la ingeniería es indispensable para construir un ciberespacio resiliente frente a amenazas y vectores de ataque avanzados.

7. Conclusiones

- La implementación del "Juego del Ahorcado" mediante una arquitectura de tres capas demostró de manera práctica que el aislamiento de responsabilidades lógicas optimiza significativamente la legibilidad, la depuración y el mantenimiento del código fuente, superando los esquemas secuenciales monolíticos.
- La correcta articulación de estructuras de control repetitivas e indeterminadas condicionales es fundamental para gobernar los hilos de ejecución de sistemas interactivos, permitiendo una experiencia de usuario fluida y libre de desbordamientos lógicos en la consola.
- El correcto manejo y lectura de ficheros de texto planos externos mediante codificaciones estandarizadas UTF-8 evidencia la viabilidad de separar la lógica de procesamiento de la persistencia de datos, permitiendo escalar el banco de palabras del sistema sin modificar una sola línea de código lógico.
- Se ratifica que los fundamentos de la lógica de programación y el diseño de software estructurado no son conceptos abstractos y aislados, sino componentes transversales críticos que impactan directamente en la seguridad de la información, proveyendo al ingeniero en ciberseguridad las bases cognitivas para auditar, asegurar y validar sistemas informáticos complejos.